Лабораторная работа № 3.
Знакомство с html5. Элементы video и audio. Геолокация. Элемент canvas.

Цель: научиться подключать на веб-страницу аудио и видео файлы, определять местоположение пользователя; изучить возможности рисования на веб-странице.

Теория

HTML5 – это новая версия HTML. Наиболее интересные нововведения HTML5:

- поддержка видео и аудио (элементы video и audio);
- возможности рисования на веб-страницах произвольных объектов (элемент canvas);
- улучшение форм (новые значения type для <input> и множество новых элементов и атрибутов);
- добавление семантических тэгов, позволяющих сделать веб-страницы более понятными для поисковых систем, браузеров и других программ и устройств, читающих веб-страницы (элементы footer, header, nav, article и др.);
- DOM хранилища – более функциональная альтернатива cookie.

HTML5 содержит специальный тэг **<video>**, позволяющий встраивать в веб-страницы видео файлы. Атрибут **src** тэга <video> позволяет указать путь к видео файлу, атрибут **controls** отображает в плеере кнопки управления видео, а атрибуты **height** (высота) и **width** (ширина) задают размеры плеера. Например,

<video src=video.ogv" width="300" height="200" controls="controls"></video>

С помощью нового HTML5 элемента **<audio>** можно добавить на веб-страницу музыкальную композицию. Атрибут **src** тэга <audio> позволяет указать путь к аудио файлу, а атрибут **controls** добавляет кнопки управления. Например,

<audio src="stop.ogv" controls="controls"> </audio>

С помощью HTML5 геолокации можно определить местоположение пользователя.

Текущая позиция пользователя может быть определена с помощью метода **getCurrentPosition()** объекта **navigator.geolocation**.

navigator.geolocation.getCurrentPosition(success_function, error_function, options);

Параметры метода:

success_function – имя функции, которая будет вызвана в случае удачного считывания координат;

error_function – имя функции, которая будет вызвана при ошибке;

options – задает настройки, которые будут использованы при считывании координат. Возможные значения:

enableHighAccuracy – если имеет значение true, браузер будет пытаться определить местоположение как можно точнее;

timeout – устанавливает максимально допустимое время для считывания данных (по умолчанию не ограничено);

maximumAge – как долго браузер будет хранить к кэше предыдущее сохраненное значение.

Если пользователь разрешил использовать данные о его местоположении и они были удачно считаны браузером, то в функцию **success_function** в качестве параметров будет передан объект, содержащий свойство **timestamp** (время считывания координат), и объект **coords** со следующими свойствами:

- **latitude** и **longitude** – широта и долгота местоположения;
- **altitude** – высота над уровнем моря в метрах;
- **accuracy** – точность определения широты и долготы (чем больше число, тем меньше точность);
- **altitudeAccuracy** – точность определения высоты (чем больше число, тем меньше точность);
- **heading** – направление пользователя в градусах (т.е. 0 градусов значит, что пользователь направляется на север, 180 на юг и т.д.)
- **speed** – скорость перемещения в метрах в секунду.

В следующем примере определяются широта и долгота местоположения пользователя:

```
<html>

<script>

function getCoordinates() {

navigator.geolocation.getCurrentPosition(showCoordinates);

}

function showCoordinates(position) {

document.write("Широта:" + position.coords.latitude + "<br/>");

document.write("Долгота:" + position.coords.longitude);

}

</script>

<body>

<p>Для считывания координат нажмите на кнопку</p>

<input type="button" value=" Координаты" onclick =
```

"getCoordinates()">

</body>

</html>

Элемент <canvas>

Элемент<canvas>позволяет рисовать на веб-страницах произвольные фигуры с помощью JavaScript (или других клиентских скриптов). Сам по себе canvas ничего не рисует, это холст, который предоставляет возможности для рисования.Создать canvas можно так:

<canvas id='draw' width='300' height='200'>…</canvas>.

Для создания прямоугольников:

fillRect(x,y,ширина,высота) – рисует закрашенный прямоугольник;

strokeRect(x,y,ширина,высота) – не закрашенный прямоугольник;

clearRect(x,y,ширина,высота) – очищает указанную зону (x и y – величина смещения прямоугольника от верхнего левого угла холста в пикселях).

В следующем примере на экран выводятся два прямоугольника: закрашенный с очищенной зоной внутри и не закрашенный. Первые две строки являются стандартными для рисования любого объекта в canvas.

var canvas = document.getElementById("draw");

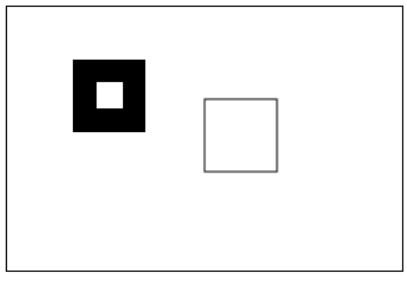
var x=canvas.getContext("2d");

x.fillRect(50,40,55,55);

x.strokeRect(150,70,55,55);

x.clearRect(68,57,20,20);

```
<html>
<body>
<canvas id='draw' width='300' height='200' style='border:1px solid'></canvas>
<script type='text/javascript'>
var canvas=document.getElementById("draw")
var x=canvas.getContext("2d");
x.fillRect(50,40,55,55);
x.strokeRect(150,70,55,55);
x.clearRect(68,57,20,20);
</script>
</body>
</html>
```



Рисование составных фигур

Составные фигуры состоят из нескольких соединенных простых объектов.

moveTo(x,y) – устанавливает координаты точки, из которой начнется рисование следующего объекта;

lineTo(x,y) – рисует прямую линию:

arc(x,y,радиус,нач_угол,конеч_угол) – рисует круг. Угол необходимо задавать в радианах, (радианы=(Math.PI/180)*градусы);

rect(x, y, ширина, высота) – рисует прямоугольник.

```
x.beginPath();

x.moveTo(20,20);

x.lineTo(70,70);

x.lineTo(20,70);

x.closePath();

x.fill();
```

Для раскрашивания нарисованных в canvas фигур предусмотрены свойства: **fillStyle** (применяется к закрашенным фигурам) и **strokeStyle** (к незакрашенным фигурам).

```
x.fillStyle="green";

x.strokeStyle="#FF45FF"

x.fillStyle="rgb(255,73,73)"

x.fillStyle="rgba(0,0,0,0.5)" //(0.5 – прозрачность)
```

Для оформления линий используются: **lineWidth** – ширина линии, **lineCap** – кончики линий (round – закругленные) , **lineJoin** – сглаживание стыков двух линий (round).

Градиент

CreateLinearGradient(x1,y1,x2,y2) – создать линейный градиент, x1 и y1 координаты начальной, x2 и y2 – конечной точки градиента. После создания градиента надо указать цвета перехода с помощью метода **addColorStop(точка,цвет).**

```
var grad =x.createLinearGradient(0,0,0,150);

grad.addColorStop(0.0,'black');

grad.addColorStop(1.0,'white');

x.fillStyle=grad;

x.fillRect(20,20,200,200);
```

Создание теней

shadowOffsetX – смещение тени от объекта по горизонтали (может быть отрицательным).

shadowOffsetY – смещение тени по вертикали (может быть отрицательным)

shadowBlur – величина размытия тени

shadowColor – цвет тени (по умолчанию черный).

```
x.shadowOffsetX=3;

x.shadowOffsetY=3;

x.shadowBlur=5;

x.shadowColor='black';

x.fillStyle='#ffaa00';

x.fillRect(50,40,55,55);
```

Текст

Метод **fillText("текст",x,y)**позволяет отображать в элементе canvas произвольный текст, который может быть оформлен с помощью свойствafont.

```
x.font='15px Verdana';

x.fillStyle='#60016d';

x.fillText("Я студент БГТУ", 10, 40);
```

Вставка изображений

В canvas могут быть вставлены изображения форматов PNG, GIF и JPEG с помощью метода **drawImage('ссылка на картинку',x,y)**

```
<img id='image1' src='ris.jpg' style='display:none;'/>

<canvas id='draw' width='400' height='300' style='border:1px solid'></canvas>

<script>

Var img=document.getElementById('image1');

var canvas=document.getElementById("draw")

var x=canvas.getContext("2d");

x.drawImage(img,10,10);

</script>
```

Задания для лабораторной работы № 3

Задание 1. Создать новую веб-страницу. Вставит заголовок в центре страницы «Я, ФИО, изучаю HTML5». Подсветить ФИО с помощью тега <mark>.

Задание 2. Вставить аудио и видео файлы.

Задание 3. Определить свои координаты с помощью геолокации.

Задание 4. Нарисовать любую картинку (можно домик), используя простые фигуры. Применить возможности для оформления – цвет, тени, градиент. Созданную картинку подписать (свои ФИО).